知識表現と推論

(knowledge representation and reasoning)

Unit 17　三つのモデル (three models)

宋 剛秀 (そう たけひで)

名古屋大学

この unit の内容

- **内容**：N クイーン，数独，グラフ彩色
- **前の unit**：Unit 16　ソルバーに渡すもの
- **次の unit**：Unit 18　モデルの比較

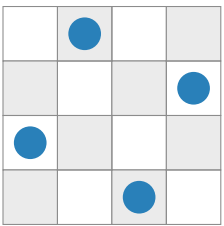
この unit の到達点

N クイーン，数独，グラフ彩色を，Syrup のモデルとして書ける．

N クイーン：変数の取り方

$N \times N$ の盤に N 個のクイーンを，どの二つも同じ行・列・斜めに並ばないように置く (Unit 09)．

- **変数**：行 R のクイーンの列を $q_R \in [1, N]$ とする．
- どの行にもクイーンはちょうど一つある．これは変数の取り方に入っている．
- 残りの条件は三つである．
  - 同じ列にない
  - 右下がりの斜めにない
  - 右上がりの斜めにない



$q = (2, 4, 1, 3)$

Unit 09 の q_i は「列 i の行」だった．ここでは行と列を入れ替えている．どちらでも同じ問題である．

N クイーン：モデル 1 (二つずつの制約)

どの二つの行 $R < S$ にも，Unit 09 と同じ制約を書く．

```
row(1..4).
(int q[R] 1 4) :- row(R).
(!= q[R] q[S]) :- row(R), row(S), R < S.
(!= (- q[S] q[R]) D) :- row(R), row(S), R < S, D = S - R.
(!= (- q[R] q[S]) D) :- row(R), row(S), R < S, D = S - R.
```

- 3 **行目は「同じ列にない」，**4・5 **行目は「斜めにない」である．**
- `D = S - R` **は，行の差を計算して** D **に入れる．**
- 4 **クイーンでは，**$\neq$ **の制約が** 18 **個できる．**Sugar **では** 22 **行である．**

N クイーン：モデル 1 が生成する Sugar

4 クイーンでは，次の 22 行が作られる．

```
(int q_1 1 4)
(int q_2 1 4)
(int q_3 1 4)
(int q_4 1 4)
(!= q_1 q_2)
(!= q_1 q_3)
(!= q_1 q_4)
(!= q_2 q_3)
(!= q_2 q_4)
(!= q_3 q_4)
```

```
(!= (- q_2 q_1) 1)
(!= (- q_3 q_1) 2)
(!= (- q_4 q_1) 3)
(!= (- q_3 q_2) 1)
(!= (- q_4 q_2) 2)
(!= (- q_4 q_3) 1)
```

```
(!= (- q_1 q_2) 1)
(!= (- q_1 q_3) 2)
(!= (- q_1 q_4) 3)
(!= (- q_2 q_3) 1)
(!= (- q_2 q_4) 2)
(!= (- q_3 q_4) 1)
```

左が宣言と「同じ列にない」，中と右が二つの向きの「斜めにない」である．

N クイーン：斜めを式で書く

同じ斜めの線の上のマスは，「列 $+$ 行」か「列 $-$ 行」が等しい．

	列 1	列 2	列 3	列 4
行 1	2, 0	3, 1	4, 2	5, 3
行 2	3, -1	4, 0	5, 1	6, 2
行 3	4, -2	5, -1	6, 0	7, 1
行 4	5, -3	6, -2	7, -1	8, 0

- 各マスに「列 $+$ 行，列 $-$ 行」を書いた．右上がりの斜めは前の数，右下がりの斜めは後ろの数が等しい．
- クイーンのマスは (行 R，列 q_R) である．斜めの条件は「$q_R + R$ がすべて違う」「$q_R - R$ がすべて違う」になる．

N クイーン：モデル 2 (alldifferent)

三つの条件を，それぞれ `alldifferent` **一つで書く．**

```
row(1..4).
(int q[R] 1 4) :- row(R).
(alldifferent (seq q[R] : row(R)))
(alldifferent (seq (+ q[R] R) : row(R)))
(alldifferent (seq (- q[R] R) : row(R)))
```

- **制約は** 3 **個だけである．盤の大きさによらない．**
- `1..4` **と** `1 4` **を** `1..8` **と** `1 8` **にすれば，**8 **クイーンになる．**
- **解は** $q = (2, 4, 1, 3)$ **である．**8 **クイーンの解は全部で** 92 **通りある．**

N クイーン：モデル 3 (0-1 変数)

マスごとに「クイーンがあるか」を 0-1 変数 $b_{R,C}$ で表すこともできる．

```
row(1..4). col(1..4).
(int b[R,C] 0 1) :- row(R), col(C).
(= (+ (seq b[R,C] : col(C))) 1) :- row(R).
(= (+ (seq b[R,C] : row(R))) 1) :- col(C).
(<= (+ (seq b[R,C] : row(R), col(C), R + C = S)) 1) :- S = 3..7.
(<= (+ (seq b[R,C] : row(R), col(C), R - C = D)) 1) :- D = -2..2.
```

- 各行と各列で，和がちょうど 1 である．
- 各斜めの線で，和が 1 以下である．マスが一つしかない隅の斜めは省いた．
- 「各行に一つ」は，モデル 1・2 では変数の取り方に入っていたが，ここでは制約で書く．

N クイーン：三つのモデルの大きさ

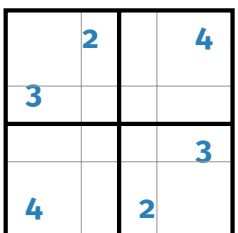
ソルバーの統計を並べる．どのモデルでも同じ解が見つかる．

	4 クイーン		8 クイーン	
モデル	変数	制約	変数	制約
1：二つずつの $\neq$	4	18	8	84
2：`alldifferent`	4	3	8	3
3：0-1 変数	16	18	64	42

- モデル 1 の制約は，盤の大きさの 2 乗で増える．モデル 2 は 3 個のままである．
- モデル 3 は，変数がマスの数だけ要る．
- どれが速いかは，ソルバーの解き方による．Unit 18 で考える．

数独：問題

4×4 の数独を考える．各行・各列・各 2×2 ブロックに，1 から 4 が一つずつ入る．



- 青い数字がヒントである．
- 変数は，マス（行 R，列 C）の数 $x_{R,C} \in [1, 4]$ とする．
- 制約は三種類である．
  - ヒントのマスは，その値に等しい．
  - 各行，各列は `alldifferent`．
  - 各ブロックは `alldifferent`．

数独：変数とヒント

```
row(1..4). col(1..4).
hint(1,2,2). hint(1,4,4). hint(2,1,3).
hint(3,4,3). hint(4,1,4). hint(4,3,2).
(int x[R,C] 1 4) :- row(R), col(C).
(= x[R,C] V) :- hint(R,C,V).
```

- `hint(R,C,V)` は「マス (R, C) のヒントは V」というファクトである．
- 4 行目で 16 個の変数 $x_{1,1}, \ldots, x_{4,4}$ を宣言する．名前は `x_1_1` のようになる．
- 5 行目は，ヒントごとに `(= x_1_2 2)` のような制約を作る．
- 問題を変えるときは，`hint` のファクトだけを書き換える．

数独：行と列

```
(alldifferent (seq x[R,C] : col(C))) :- row(R).
(alldifferent (seq x[R,C] : row(R))) :- col(C).
```

- 1 行目は，外側の条件 `row(R)` で行を一つ決め，内側の `seq` でその行のマスを並べる．
- 行 1 については `(alldifferent x_1_1 x_1_2 x_1_3 x_1_4)` が作られる．
- 2 行目は，同じことを列について行う．

`seq` の中の変数 `(C)` は並べるために回り，外の変数 `(R)` は制約ごとに一つ決まる．

数独：生成される制約

行と列の 2 行のルールから，次の 8 個の制約が作られる．

```
(alldifferent x_1_1 x_1_2 x_1_3 x_1_4)
(alldifferent x_2_1 x_2_2 x_2_3 x_2_4)
  ...
(alldifferent x_1_1 x_2_1 x_3_1 x_4_1)
(alldifferent x_1_2 x_2_2 x_3_2 x_4_2)
  ...
```

- 1 本目のルールから 4 個 (行)，2 本目から 4 個 (列) である．
- 変換の結果を見ると，ルールが思ったとおりに働いているかを確かめられる．

数独：ブロックを作る

マス (R, C) がブロック (P, Q) に入るという関係を，ルールで作る．

```
block(R,C,P,Q) :- P = 0..1, Q = 0..1, I = 1..2, J = 1..2,
                  R = P * 2 + I, C = Q * 2 + J.
(alldifferent (seq x[R,C] : block(R,C,P,Q))) :- P = 0..1, Q = 0..1.
```

- ルールの左側は S 式でなくてもよい．`block` という新しいファクトが 16 個できる．
- ブロック (P, Q) は，行 $2P + 1$, $2P + 2$ と列 $2Q + 1$, $2Q + 2$ からなる．
- 2 行目は，ブロックごとに `alldifferent` を作る．

数独：解

ソルバーは SAT を返す．解は次のとおりである．

1	**2**	3	**4**
3	4	1	2
2	1	4	**3**
4	3	**2**	1

- 変数 16 個，制約 18 個である．
  - ヒントが 6 個，`alldifferent` が 12 個 (行 4，列 4，ブロック 4)
- この問題は，探索に入る前の伝播だけで解けた．統計では，SAT の変数が 0 個である．

数独：9 × 9 へ広げる

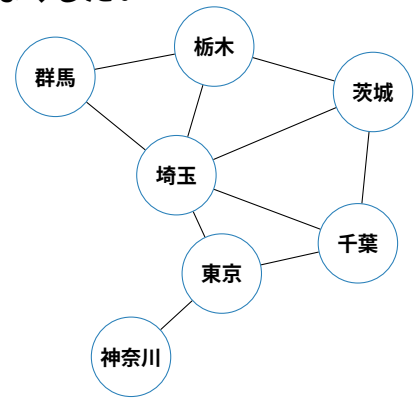
変えるのは，範囲とブロックの大きさと，ヒントだけである．

```
row(1..9). col(1..9).
(int x[R,C] 1 9) :- row(R), col(C).
block(R,C,P,Q) :- P = 0..2, Q = 0..2, I = 1..3, J = 1..3,
                  R = P * 3 + I, C = Q * 3 + J.
(alldifferent (seq x[R,C] : block(R,C,P,Q))) :- P = 0..2, Q = 0..2.
```

- **行と列の** `alldifferent` **の** 2 **行は，そのまま使える．**
- **制約は，** `alldifferent` **が** 27 **個とヒントの数だけになる．**
- **難しい問題では伝播だけでは決まらず，探索が要る．ヒント** 21 **個の難問では，探索に入ってから解が見つかった．**

グラフ彩色：関東の地図

関東の 7 都県を，隣り合う都県が違う色になるように塗る．色はできるだけ少なくしたい．



- 頂点が都県，辺が「隣り合う」である．
- 辺は 10 本ある．
- 茨城・埼玉・千葉は互いに隣り合う．三角形がある．

グラフ彩色：ファクトと変数

```
border(ibaraki,tochigi). border(ibaraki,saitama).
border(ibaraki,chiba).   border(tochigi,gunma).
border(tochigi,saitama). border(gunma,saitama).
border(saitama,chiba).   border(saitama,tokyo).
border(chiba,tokyo).     border(tokyo,kanagawa).
pref(P) :- border(P,_).
pref(P) :- border(_,P).
(int c[P] 1 4) :- pref(P).
(!= c[P] c[Q]) :- border(P,Q).
```

- 地図をファクトで書き，都県の一覧 `pref` をルールで導く．
- 色は 1 から 4 の整数とする．隣り合う都県は違う色である．

グラフ彩色：2 色では塗れない

`(int c[P] 1 4)` を `(int c[P] 1 2)` にすると，ソルバーは UNSAT を返す．

- 茨城・埼玉・千葉の三角形は，2 色では塗れない．
- Unit 08 と Unit 12 の「三つの地域と二つの色」と同じ構造が，地図の中にある．

- **UNSAT から分かること**
  - 2 色で塗る方法は一つもない．
  - 色の数の下限は 3 である．
- **UNSAT から分からないこと**
  - どの部分が原因かは，出力からは分からない．三角形は，人が見つけた．

グラフ彩色：色の数を最小にする

使う色の数を変数 k にし，それを最小にする．

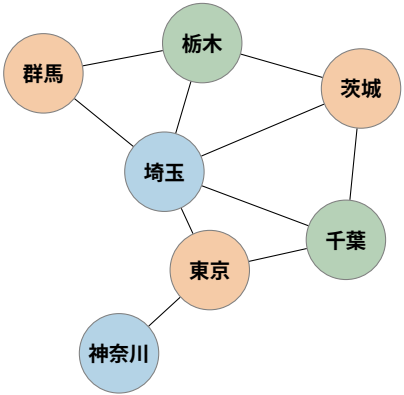
```
(int k 1 4)
(<= c[P] k) :- pref(P).
(objective minimize k)
```

- どの都県の色も k 以下である．k は「使ってよい色の数」になる．
- `objective` があるので，ソルバーは k がいちばん小さい解を探す．

ソルバーの出力

最適解が見つかった．$k = 3$．

グラフ彩色：結果



色	都県
1（青）	埼玉，神奈川
2（緑）	栃木，千葉
3（橙）	群馬，茨城，東京

3 色で塗れた．2 色では塗れないので，3 が最小である．

UNSAT の原因を探す

ソルバーは，UNSAT の理由までは教えてくれない．原因は，モデルを小さくして探す．

1. 制約やファクトを一部だけ外して，もう一度解く．
2. SAT になったら，外したものの中に原因がある．UNSAT のままなら，残りの中にある．
3. 原因の範囲が小さくなるまで，くり返す．

- 例．関東の地図を 2 色で塗る (UNSAT)．
  - `border(ibaraki,chiba).` を外しても UNSAT のままである．三角形はほかにもある．
  - 茨城・埼玉・千葉の間の 3 本だけを残しても UNSAT である．この三角形だけで矛盾が起きる．
- 自分のモデルの誤りを探すときも同じである．ありえない UNSAT は，制約の書き過ぎを疑う．

試験の日程を彩色で解く

生徒が受ける科目から，同じ日にできない科目の組を導く．

```
takes(aoi,math). takes(aoi,english). takes(aoi,music).
  ... (5 人分)
subject(A) :- takes(_,A).
clash(A,B) :- takes(S,A), takes(S,B), A < B.
(int d[A] 1 5) :- subject(A).
(!= d[A] d[B]) :- clash(A,B).
(int k 1 5)
(<= d[A] k) :- subject(A).
(objective minimize k)
```

- `clash` **が辺，**`d[A]` **が色 (試験日) である．**11 **組の** `clash` **ができる．**
- **最適値は** $k = 4$ **である．数学・英語・理科・美術が互いにぶつかるので，**4 **日より少なくはできない．**

同じ形の問題

グラフ彩色の形をした問題は多い．頂点と辺の意味を読み替えるだけで，同じモデルが使える．

問題	頂点	辺（違う色にする組）
地図の塗り分け	地域	隣り合う
試験の日程	科目	同じ生徒が受ける
レジスタ割当て	変数	同時に生きている
周波数の割当て	送信局	近くて干渉する

色は，日付，レジスタ，周波数に当たる．色の数の最小化は，日数やレジスタ数の最小化である．